

Mean

```
In [1]: marks = [45, 50, 55, 60, 90]
mean = sum(marks) / len(marks)
print("Mean:", mean)
```

Mean: 60.0

Median

```
In [2]: import statistics
marks = [45, 50, 55, 60, 90]
median = statistics.median(marks)
print("Median:", median)
```

Median: 55

Mode

```
In [3]: import statistics
data = [10, 20, 20, 30, 40]
mode = statistics.mode(data)
print("Mode:", mode)
```

Mode: 20

```
In [4]: import pandas as pd
df = pd.DataFrame({
    'Marks': [45, 50, 55, 60, 90]
})
print("Mean:", df['Marks'].mean())
print("Median:", df['Marks'].median())
print("Mode:", df['Marks'].mode())
```

Mean: 60.0

Median: 55.0

Mode: 0 45

1 50

2 55

3 60

4 90

Name: Marks, dtype: int64

Variance

```
In [6]: import numpy as np
data = [10, 12, 14, 16, 18]
print("Variance:", np.var(data))
```

Variance: 8.0

Standard Deviation

```
In [8]: import numpy as np
data = [10, 12, 14, 16, 18]
print("Variance:", np.var(data))
print("Standard Deviation:", np.std(data))
```

Variance: 8.0
Standard Deviation: 2.8284271247461903

IQR

```
In [9]: import numpy as np
data = [10,20,30,40,50,60]
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
```

Q1: 22.5
Q3: 47.5
IQR: 25.0

Outlier Detection

```
In [15]: import numpy as np
data = [10,20,30,40,50,60,1000]
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print("Lower:", lower)
print("Upper:", upper)
outliers = [x for x in data if x < lower or x > upper]
print("Outliers:", outliers)
```

Lower: -20.0
Upper: 100.0
Outliers: [1000]

Probability

```
In [17]: sample_space = {1, 2, 3, 4, 5, 6}
event_A = {2, 4, 6} #Even Numbers
print("Sample Space(S):", sample_space)
print("Event A:", event_A)
#2.probability P(A)
P_A = len(event_A) / len(sample_space)
print("\nProbability of Event A")
print("P(A):", P_A)
#3.Conditional Probability
#P(A/B)
event_B = {4,5,6}
intersection = event_A.intersection(event_B)
P_A_given_B = len(intersection) / len(event_B)
print("\nEvent B:", event_B)
print("A intersection B:", intersection)
print("P(A|B) =", P_A_given_B)
```

Sample Space(S): {1, 2, 3, 4, 5, 6}
Event A: {2, 4, 6}

Probability of Event A
 $P(A) = 0.5$

Event B: {4, 5, 6}
A intersection B: {4, 6}
 $P(A|B) = 0.6666666666666666$

```
In [18]: # Bayes Theorem
#Example:
#Disease Prevalence = 1%
#Test Sensitivity = 99%
P_Disease = 0.01
P_Positive_given_Disease = 0.99
P_Positive = 0.05
P_Disease_given_Positive = (
    P_Positive_given_Disease * P_Disease
) / P_Positive
print("\nBayes Theorem Example")
print("P(Disease) =", P_Disease)
print("P(Positive|Disease) =", P_Positive_given_Disease)
print("P(Positive) =", P_Positive)
print(
    "P(Disease|Positive) =",
    round(P_Disease_given_Positive, 4)
)
print("Percentage =", round(P_Disease_given_Positive * 100, 2), "%")
```

Bayes Theorem Example
 $P(\text{Disease}) = 0.01$
 $P(\text{Positive}|\text{Disease}) = 0.99$
 $P(\text{Positive}) = 0.05$
 $P(\text{Disease}|\text{Positive}) = 0.198$
Percentage = 19.8 %

Correlation

```
In [21]: import pandas as pd
df = pd.DataFrame({
    'Hour': [1,2,3,4,5],
    'Marks': [20,40,50,70,90]
})
corr = df['Hour'].corr(df['Marks'])
print("Pearson Correlation:", corr)
#Close to +1
#Strong positive relation
```

Pearson Correlation: 0.9948497511671097

```
In [22]: import pandas as pd
df = pd.DataFrame({
    'Rank_Study': [1,2,3,4,5],
    'Rank_Marks': [2,1,3,5,4]
})
corr = df['Rank_Study'].corr(
    df['Rank_Marks'],
    method = 'spearman'
```

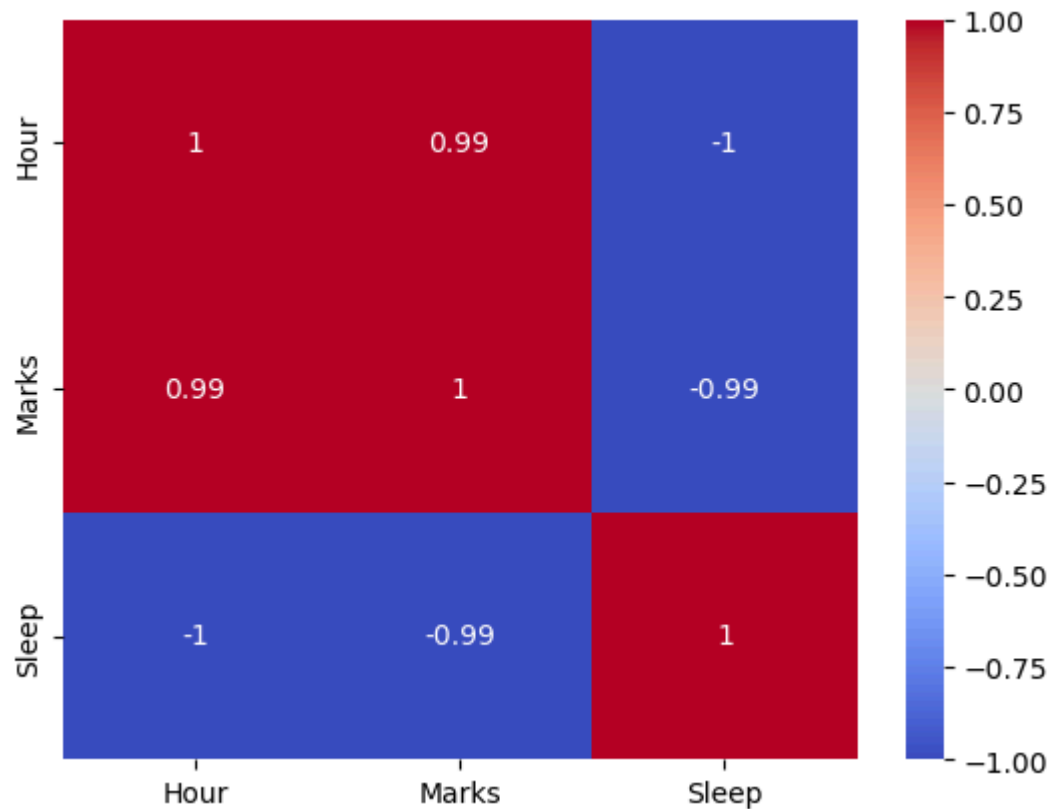
```
)
print("Spearman Correlation:", corr)
```

Spearman Correlation: 0.7999999999999999

```
In [23]: #Correlatio Matrix
import pandas as pd
df = pd.DataFrame({
    'Hour': [1,2,3,4,5],
    'Marks': [20,40,50,70,90],
    'Sleep': [8,7,6,5,4]
})
print(df.corr())
```

	Hour	Marks	Sleep
Hour	1.00000	0.99485	-1.00000
Marks	0.99485	1.00000	-0.99485
Sleep	-1.00000	-0.99485	1.00000

```
In [41]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.DataFrame({
    'Hour': [1,2,3,4,5],
    'Marks': [20,40,50,70,90],
    'Sleep': [8,7,6,5,4]
})
sns.heatmap(df.corr(),
            annot=True,
            cmap='coolwarm')
plt.show()
```



Correlation vs Causation

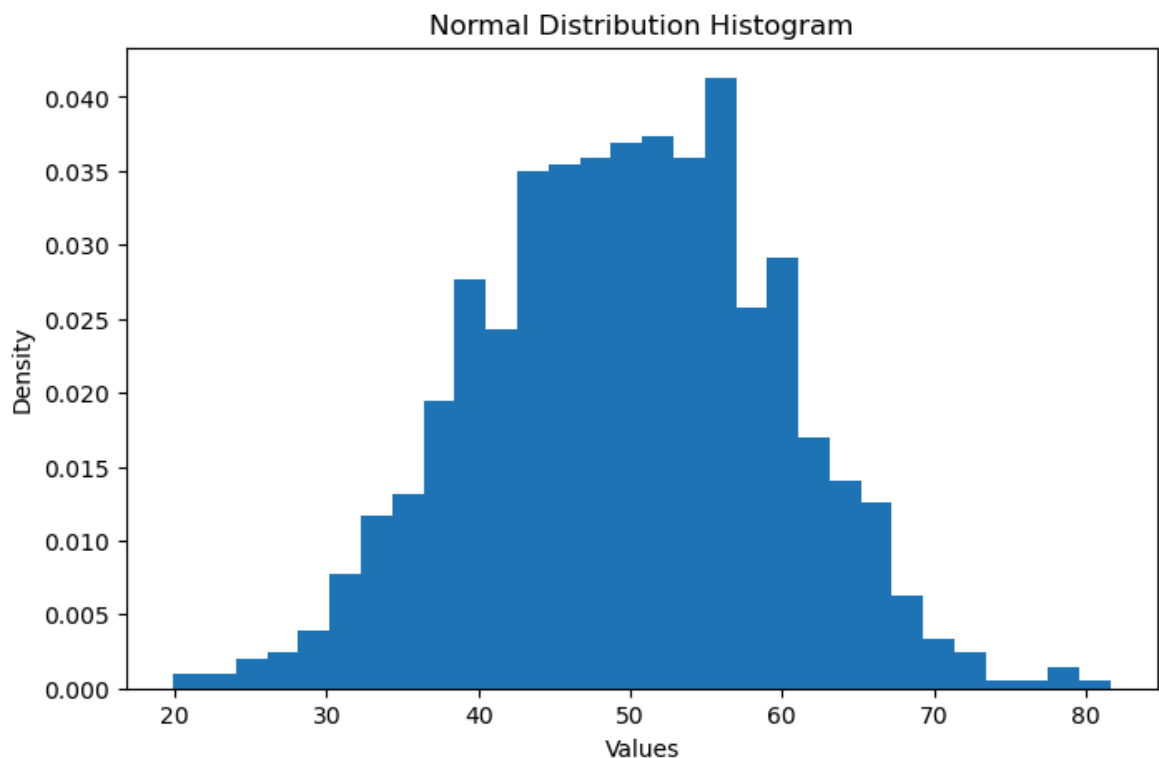
```
In [13]: #normal distribution
import numpy as np
import matplotlib.pyplot as plt

# Generate random data
data = np.random.normal(loc=50, scale=10, size=1000)

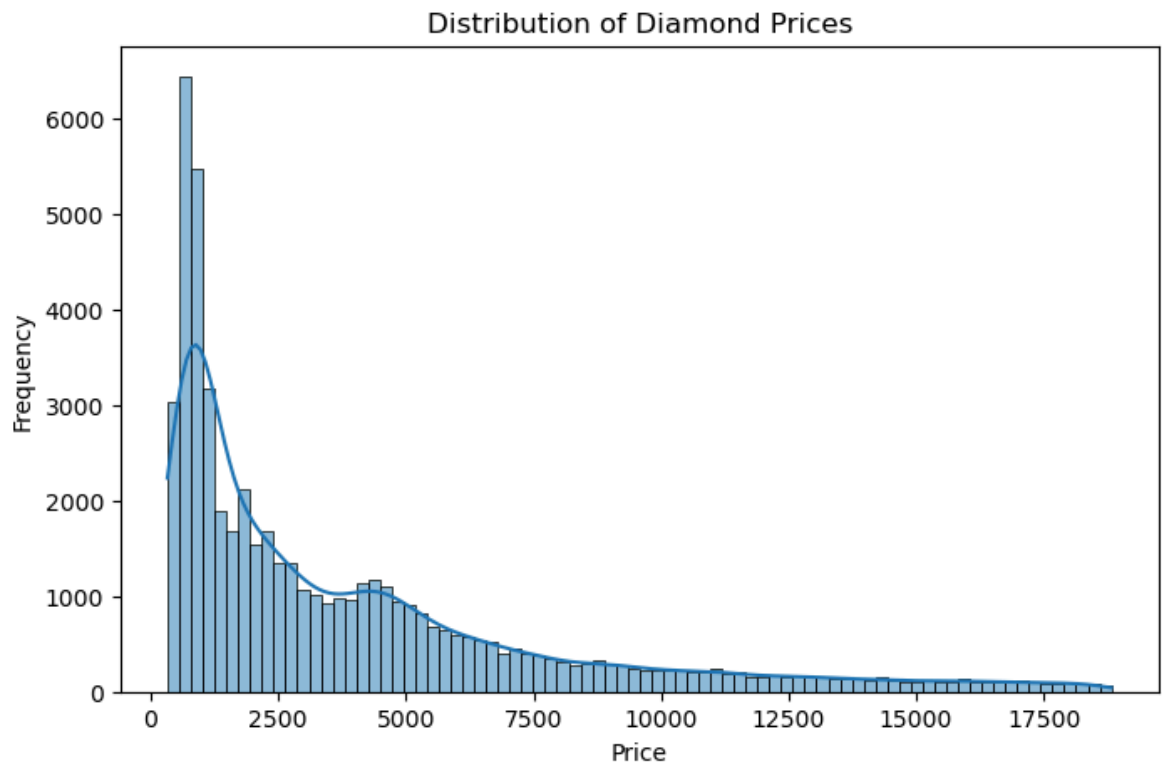
# Plot histogram
plt.figure(figsize=(8,5))
plt.hist(data, bins=30, density=True)

plt.title("Normal Distribution Histogram")
plt.xlabel("Values")
plt.ylabel("Density")

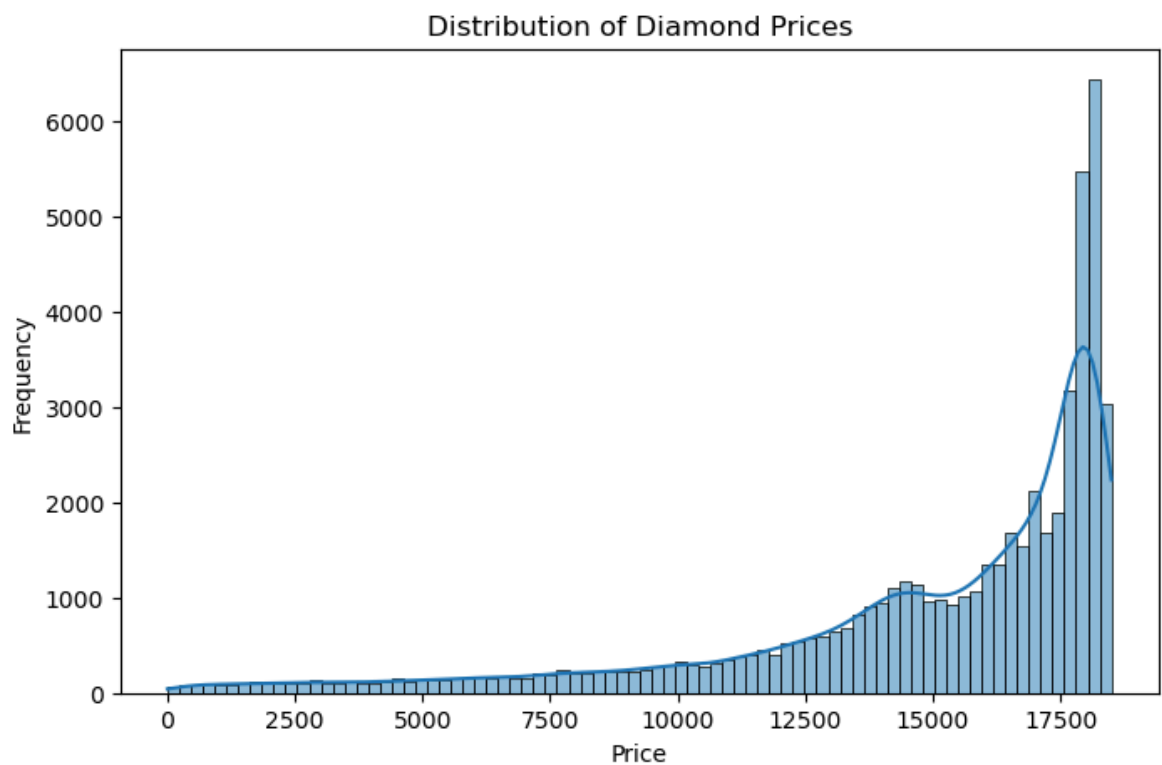
plt.show()
```



```
In [9]: import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset("diamonds")
plt.figure(figsize=(8,5))
sns.histplot(df['price'], kde=True)
plt.title("Distribution of Diamond Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```



```
In [14]: import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset("diamonds")
left_skew = df["price"].max() - df["price"]
plt.figure(figsize=(8,5))
sns.histplot(left_skew, kde=True)
plt.title("Distribution of Diamond Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
print("Skewness:", left_skew.skew())
```



Skewness: -1.6183952833835282

```
In [16]: import numpy as np
scores = [60,65,70,75,80]
mean = np.mean(scores)
std = np.std(scores)
score = 80
z_score = (score - mean) / std
print("Mean:", mean)
print("Standard Deviation:", std)
print("Z-score:", z_score)
```

Mean: 70.0

Standard Deviation: 7.0710678118654755

Z-score: 1.414213562373095

```
In [ ]: (
```